hashlock.

Security Audit

K613 (DeFi)

April 2026 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The K613 team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

K613 is a DeFi lending and staking protocol built on high performance infrastructure, with yield designed to come from real onchain lending activity rather than unsustainable emissions. Its core lending layer generates returns through genuine user demand across supply and borrow markets, creating a direct connection between protocol usage and value creation.

The protocol also includes a staking system centered on xK613, a 1:1 non rebasing receipt token that represents ownership in the protocol. Users stake K613 to receive xK613 and earn rewards funded by lending revenue and liquidity provisioning. Early exit penalties and a structured exit queue are designed to support long term alignment and reduce short term sell pressure.

K613's modular architecture separates lending, staking, and treasury functions, allowing for clearer accounting and more flexible capital management. Treasury operations, including buybacks and targeted revenue allocation, are intended to strengthen value accrual for stakers. Overall, K613 is positioned as a scalable DeFi protocol moving from incentive driven growth toward a more sustainable, revenue backed yield model.

Project Name: K613

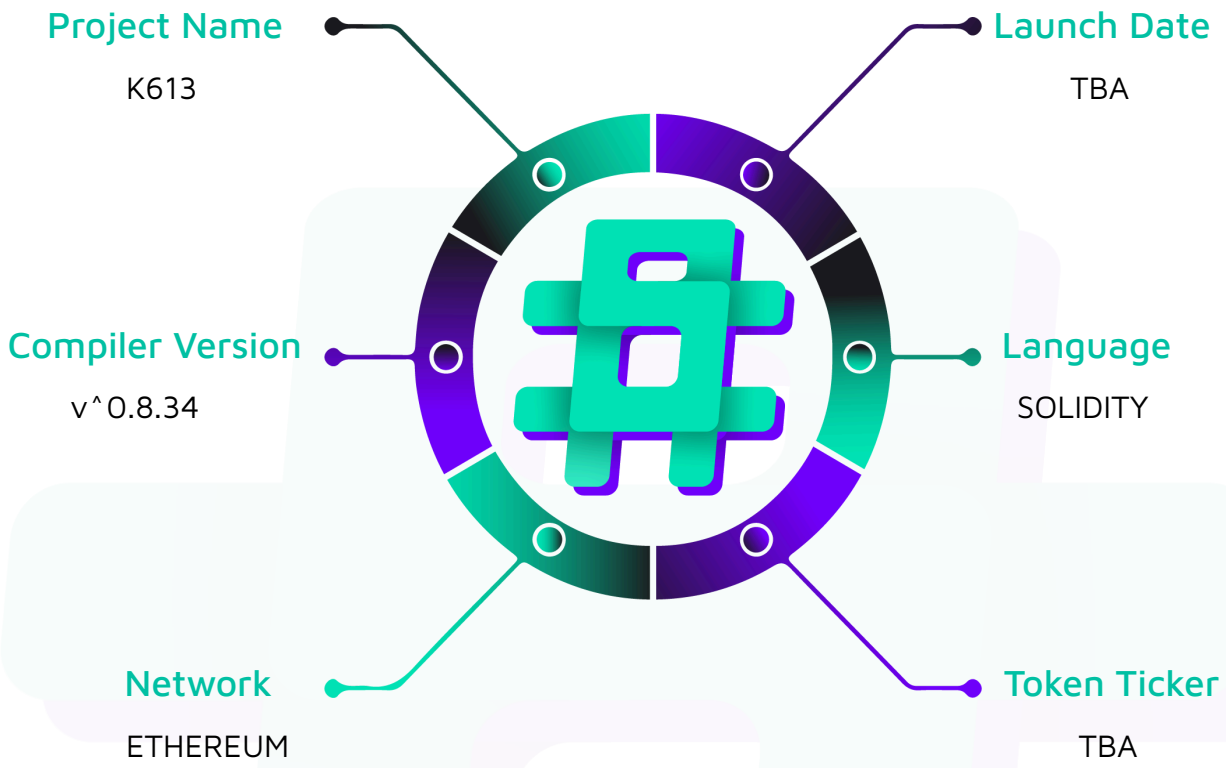
Project Type: DeFi

Compiler Version: ^0.8.34

Website: https://k613.net/

Logo:

Visualised Context:



Audit Scope

We at Hashlock audited the solidity code within the K613 project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	**K613 Smart Contracts**
Network	**Ethereum**
Language	**Solidity**
Audit Date	**April, 2026**
Contract 1	Staking/RewardsDistributor.sol
Contract 2	Staking/Staking.sol
Contract 3	Treasury/Treasury.sol
Audited GitHub Commit Hash	8f24dfa398e4977330a11c0f9f47cddcc9aa5b49
Fix Review GitHub Commit Hash	cc903434769378c05b809f5aafaa41d0e9208b67

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The list of audited assets is presented in the Audit Scope section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High severity vulnerability

2 Medium severity vulnerabilities

4 Low severity vulnerabilities

2 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour

RewardsDistributor.sol
- Allows the admin to:
  - Set the staking contract (auto-grants notifier role)
- Allows the pauser to:
  - Pause / unpause
- Allows the notifier role (Staking, Treasury) to:
  - Notify xK613 rewards
  - Add pending K613 penalties
- Allows users to:
  - Deposit / withdraw xK613
  - Claim rewards
  - Advance the epoch
	Contract achieves this functionality.

Staking.sol
- Allows the admin to:
  - Set the rewards distributor
- Allows the pauser to:
  - Pause / unpause
- Allows users to:
  - Stake K613 for xK613 (1:1)
  - Initiate, cancel, or execute a queued exit
  - Instant-exit before lock with a BPS penalty
	Contract achieves this functionality.

Treasury.sol
- Allows the admin to:
  - Deposit K613 as rewards
  - Manage the router whitelist
	Contract achieves this functionality.

- Execute Uniswap V3 buybacks - Withdraw any ERC20 - Approve xK613 pulls - Allows the pauser to: - Pause / unpause	

Code Quality

This audit scope involves the smart contracts of the K613 project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the K613 project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.
Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-severity issues should be resolved before deployment. While they do not typically lead to a complete loss of funds, they may result in partial loss of funds or unintended behavior under certain conditions.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] RewardsDistributor & Treasury - Reward xK613 tokens are permanently non-redeemable for underlying K613

Description

The protocol stakes K613 on behalf of `RewardsDistributor` (via `_stakeHeldK613`) and `Treasury` (via `depositRewards`), then distributes the resulting xK613 to users as rewards. Users who receive these reward xK613 tokens cannot convert them back to K613 through `Staking`.

Vulnerability Details

`Staking.initiateExit()` enforces `amount <= s.amount - inQueue`, where `s.amount` is the caller's own original staking position. Reward xK613 transferred to a user does not increase `_userState[user].amount`, and neither `RewardsDistributor` nor `Treasury` exposes any function to call `initiateExit()` / `exit()` on `Staking`. The K613 backing reward xK613 therefore stays locked under the system contracts' staking positions forever.

This contradicts the contract NatSpec at `RewardsDistributor.sol:16` which states that rewards can be converted to K613 via Staking instant exit.

Proof of Concept

1. Alice stakes 1000 K613, deposits 1000 xK613 into RewardsDistributor
2. Bob does instantExit(1000) -> 500 K613 penalty to RewardsDistributor
3. Epoch advances, Alice claims 500 xK613 reward, withdraws 1000 xK613
4. Alice now holds 1500 xK613 but staking position is still 1000

Impact

- All penalty K613 staked by `RewardsDistributor._stakeHeldK613()` is permanently locked.
- All K613 staked by `Treasury.depositRewards()` is permanently locked.
- Reward xK613 has no path back to K613, breaking the protocol's core 1:1 backing promise for reward holders.
- xK613 is transfer-restricted (whitelist), so secondary-market exit is also blocked.

Recommendation

Implement a redemption path for reward xK613. Options:

- Add a `redeemRewards(uint256 amount)` function to `Staking` that burns the caller's xK613 and releases K613 from the `RewardsDistributor` / `Treasury` staking positions, decrementing `_totalBacking` and the system contract's `_userState.amount`.
- Add admin-gated `initiateExit` / `exit` wrappers on `RewardsDistributor` and `Treasury`.
- Distribute rewards in K613 directly instead of xK613.

Status

Resolved

[H-02] Staking#redeemRewards - Anyone holding xK613 can skip the exit penalty and drain the reward pool

Description

The new `redeemRewards()` function is meant for users who received `xK613` as rewards, so they can turn that `xK613` back into K613. The problem is the function never checks where the caller's xK613 came from. xK613 is just an `ERC20`, the xK613 you get from staking looks identical to the xK613 you get as a reward.

So any staker can use this function to exit their own stake instantly, with zero penalty, skipping the normal `instantExit` that would cost them a 50% fee.

Vulnerability Details

Inside `redeemRewards()`, when a user submits some amount of xK613:

1. The function does not touch the caller's own `_userState.amount`.
2. It reduces the backing of the system stakers (`RewardsDistributor` / `Treasury`) instead.
3. It burns the xK613 and sends `amount` K613 straight to the caller.

Comparison for a normal staker with 1000 xK613:

- `exit` (lock) -> 7 days -> none ->1000
- `instantExit` -> ~instant -> 50% -> 500
- `redeemRewards` -> instant -> none -> 1000

`redeemRewards` is strictly better than `instantExit` (saves the 500 K613 fee) and strictly better than `exit` (no 7-day wait). Rational stakers will always pick it.

Impact

- Stakers can fully bypass the 50% instant-exit penalty.
- The exit queue loses all meaning, there is no reason to wait 7 days.
- Reward backing is drained by stakers instead of flowing to reward holders.

- Legitimate reward xK613 holders become unable to redeem once the pool is empty, which is exactly the problem H-01 was supposed to fix.

Recommendation

Pick one (or more) of these:

- Charge the same penalty in `redeemRewards` that `instantExit` charges, so there is no advantage to using one over the other.
- Track reward xK613 balances per user (e.g. mint reward xK613 through a tagged path) and only let users redeem their own reward balance.
- Limit redemption to a pro-rata share of the system backing so no single user can drain it all.
- At a minimum, cap how much a single address can pull from `redeemRewards` per epoch.

Status

Resolved

Medium

[M-01] RewardsDistributor#claim - Paused Staking contract creates DoS on reward claims

Description

`RewardsDistributor.claim()` (and `advanceEpoch()`) call `_stakeHeldK613()`, which calls `IStaking(staking).stake(balance)`. `Staking.stake` carries the `whenNotPaused` modifier. If `Staking` is paused while the distributor holds any K613 from pending penalties, every `claim()` reverts.

Vulnerability Details

```
function claim() external nonReentrant whenNotPaused {
    _updateUser(msg.sender);
    uint256 reward = userPendingRewards[msg.sender];
    ...
    userPendingRewards[msg.sender] = 0;
    _stakeHeldK613();              // calls staking.stake() -> reverts when paused
    rewardToken.safeTransfer(msg.sender, reward);
}
```

A pause on `Staking` (intended for emergencies on the staking side) silently disables reward claims and epoch advancement on `RewardsDistributor`.

Impact

- Users cannot claim earned rewards while `Staking` is paused and any K613 is held by the distributor.
- `advanceEpoch()` also reverts, blocking penalty distribution.
- The DoS persists for the entire pause window, regardless of whether the pause was related to the distributor.

Recommendation

Make the staking step non-fatal so an unrelated pause does not cascade into reward DoS:

```
function _stakeHeldK613() internal {
    if (address(staking) == address(0)) return;
    uint256 balance = k613.balanceOf(address(this));
    if (balance < 1) return;
    k613.forceApprove(address(staking), balance);
    try IStaking(staking).stake(balance) {} catch {
        k613.forceApprove(address(staking), 0);
        return;
    }
    k613.forceApprove(address(staking), 0);
}
```

Status

Resolved

[M-02] RewardsDistributor#advanceEpoch - Epoch tracking becomes permanently stale when no penalties exist

Description

`lastEpochFlushAt` is only updated inside the `shouldFlushPenalties` branch of `_distributePending()`. If multiple epochs pass with no penalties, the timestamp is never advanced, and `epochPassed` stays `true` forever. As soon as any penalty arrives — even one below `MIN_PENALTY_FLUSH` — it is immediately flushed, defeating the dust threshold.

Vulnerability Details

```
bool epochPassed = block.timestamp >= lastEpochFlushAt + epochDuration;
bool shouldFlushPenalties = pendingPenalties >= MIN_PENALTY_FLUSH
    || (pendingPenalties > 0 && epochPassed);

if (shouldFlushPenalties) {
    ...
    if (epochPassed) {
        lastEpochFlushAt = block.timestamp;   // only path that updates the marker
    }
}
```

`advanceEpoch()` runs `_distributePending()` but never updates `lastEpochFlushAt` itself, so calling it during a quiet period does not advance the marker either.

Impact

- `MIN_PENALTY_FLUSH` is permanently bypassed after any quiet period, exposing `accRewardPerShare` to dust-precision loss when `totalDeposits` is large.
- Off-chain consumers of `lastEpochFlushAt` / `EpochAdvanced` see incorrect epoch boundaries.

Recommendation

Always advance `lastEpochFlushAt` in `advanceEpoch()` once the epoch has elapsed:

```
function advanceEpoch() external nonReentrant whenNotPaused {
    _stakeHeldK613();
```

```
    if (block.timestamp < lastEpochFlushAt + epochDuration) revert EpochNotReady();
    if (totalDeposits > 0) _distributePending();
    lastEpochFlushAt = block.timestamp;
    emit EpochAdvanced(block.timestamp);
}
```

And remove the `lastEpochFlushAt = block.timestamp` write from `_distributePending()` so the marker has a single source of truth.

Status

Resolved

Low

[L-01] Staking#constructor - Accepts zero `lockDuration` and zero `instantExitPenaltyBps`

Description

`Staking`'s constructor does not validate `lockDuration` or `instantExitPenaltyBps` are non-zero.

Vulnerability Details

With `lockDuration = 0`, `exit()` can be called in the same block as `initiateExit()`, neutralising the queue, and `instantExit()` always reverts with `Unlocked()` because `block.timestamp >= req.exitInitiatedAt + 0` is always true.

With `instantExitPenaltyBps = 0`, `instantExit` becomes free, removing the economic motivation for the exit queue.

Impact

A misconfiguration at deployment can render either the lock period or the penalty mechanism useless, with no on-chain way to detect or recover beyond redeployment.

Recommendation

```
if (lockDuration_ == 0) revert InvalidLockDuration();
if (instantExitPenaltyBps_ == 0) revert InvalidBps();
```

Status

Resolved

[L-02] RewardsDistributor#notifyReward - Missing event when rewards are queued as `pendingRewards`

Description

When `notifyReward()` is called while `totalDeposits == 0`, the amount is added to `pendingRewards` and the function returns without emitting any event.

Vulnerability Details

```
function notifyReward(uint256 amount) external ... {
    if (totalDeposits == 0) {
        pendingRewards += amount;
        return;                          // no event
    }
    accRewardPerShare += (amount * PRECISION) / totalDeposits;
    emit RewardNotified(amount);
}
```

Impact

Off-chain indexers and dashboards relying on `RewardNotified` will under-report total rewards by the queued amount until they are eventually distributed.

Recommendation

Emit `RewardNotified(amount)` (or a dedicated `RewardQueued(amount)`) in the `totalDeposits == 0` path as well.

Status

Resolved

[L-03] RewardsDistributor#claim - Reverts after `_distributePending` when `ExitVestingActive`

Description

`claim()` runs `_updateUser(msg.sender)` (which calls `_distributePending()` and may flush penalties / advance `accRewardPerShare`) before checking whether the user has an active exit in `Staking`. If the check fails, the entire transaction — including the penalty flush — is rolled back.

Vulnerability Details

```
function claim() external nonReentrant whenNotPaused {
    _updateUser(msg.sender);                                    // does work
    ...
    if (IStaking(staking).exitQueueLength(msg.sender) > 0)
        revert ExitVestingActive();                             // reverts the work
}
```

Impact

- Wasted gas for the caller.
- A user with active exits cannot inadvertently be the actor that finalises a penalty flush; subsequent users must trigger it.

Recommendation

Move the exit-vesting check to the top of the function:

```
function claim() external nonReentrant whenNotPaused {
    if (address(staking) != address(0) &&
        IStaking(staking).exitQueueLength(msg.sender) > 0) revert
ExitVestingActive();
    _updateUser(msg.sender); ...
}
```

Status

Resolved

[L-04] Staking - `instantExitPenaltyBps` is immutable and cannot be updated

Description

`instantExitPenaltyBps` is set in the constructor (correctly bounded by `MAX_BASIS_POINTS`) and is never exposed via a setter. Governance has no way to tune the protocol's main exit-disincentive lever.

Vulnerability Details

```
constructor(... uint256 instantExitPenaltyBps_) {
    if (instantExitPenaltyBps_ > MAX_BASIS_POINTS) revert InvalidBps();
    ...
    instantExitPenaltyBps = instantExitPenaltyBps_;}
// no setter exists
```

Impact

- Cannot raise the penalty during a bank-run-style mass instant-exit event.
- Cannot lower the penalty if it turns out to be too punitive and is suppressing legitimate liquidity.
- The only remediation is full redeployment, which would fragment liquidity and require migrating `_userState` and exit queues.

Recommendation

```
event InstantExitPenaltyBpsUpdated(uint256 oldBps, uint256 newBps);
function setInstantExitPenaltyBps(uint256 newBps) external
onlyRole(DEFAULT_ADMIN_ROLE) {
    if (newBps > MAX_BASIS_POINTS) revert InvalidBps();
    emit InstantExitPenaltyBpsUpdated(instantExitPenaltyBps, newBps);
    instantExitPenaltyBps = newBps;}
```

Optionally enforce a stricter cap and/or a timelock so users get advance notice before changes apply against them.

Status

Resolved

QA

[Q-01] Staking#_removeExitRequest - Exit queue swap-and-pop pattern causes index instability

Description

`_removeExitRequest` removes an entry by swapping it with the last element and popping. After any exit/cancel, the indices of remaining requests can change, so previously emitted events (`ExitInitiated`, `Exited`, `ExitCancelled`) reference indices that no longer point to the same request.

Impact

UX/integration only. No on-chain security impact: each user manages their own queue and the correct amounts are used at execution time. Off-chain systems and frontends must re-fetch the queue after each modification.

Recommendation

Document the behaviour, or move to a mapping-based queue keyed by a unique exit-request ID so identifiers are stable across removals.

Status

Acknowledged

[Q-02] Staking#instantExit - Penalty rounds to zero for dust amounts

Description

`penalty = (amount * instantExitPenaltyBps) / MAX_BASIS_POINTS` rounds down. With `instantExitPenaltyBps = 5_000`, any `amount <= 1` wei produces `penalty = 0`, the user pays nothing, and the `RewardsDistributorNotSet` guard (which only triggers when `penalty > 0`) is silently skipped.

Vulnerability Details

```
uint256 penalty = (amount * instantExitPenaltyBps) / MAX_BASIS_POINTS;
uint256 payout  = amount - penalty;
if (penalty > 0 && address(rewardsDistributor) == address(0)) revert
RewardsDistributorNotSet();
```

Impact

Negligible at current parameters: gas costs for many tiny `initiateExit` + `instantExit` calls vastly exceed any avoided penalty, and very long queues self-DoS the user via linear scans in `_removeExitRequest`. The concern grows if `instantExitPenaltyBps` is ever lowered.

Recommendation

Round the penalty up, or floor it at 1 wei whenever `amount > 0` and the configured BPS is non-zero:

```
uint256 penalty = (amount * instantExitPenaltyBps + MAX_BASIS_POINTS - 1) /
MAX_BASIS_POINTS;
// or:
uint256 penalty = (amount * instantExitPenaltyBps) / MAX_BASIS_POINTS;
if (penalty == 0 && instantExitPenaltyBps > 0 && amount > 0) penalty = 1;
```

Optionally enforce a minimum `initiateExit` amount to keep dust out of the queue.

Status

Resolved

Centralisation

The K613 project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the K613 project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au